

Iris Classification API – Model Deployment & Containerization Project

This project deploys a Machine Learning Iris Classification model using FastAPI. The API provides prediction, health monitoring, and interactive Swagger documentation.

Project Features

- FastAPI-based REST API
- Iris flower prediction endpoint
- Health check endpoint
- Swagger UI documentation
- Docker-ready deployment

Endpoints

GET / Root endpoint

GET /health Health check

POST /predict Predict iris species

Swagger UI Overview



Iris Classification API

[/openapi.json](#)

API for classifying Iris flowers using a Random Forest model

default

GET	/	Read Root	⌵
GET	/health	Health Check	⌵
POST	/predict	Predict	⌵

Schemas

HTTPValidationError	>	Expand all	object
IrisFeatures	>	Expand all	object
ValidationError	>	Expand all	object

Health Endpoint

New tab

Iris Classification API - Swagger UI

localhost:8000/docs#/default/predict_predict_post

GET

/health

Health Check

Health check endpoint.

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links

POST

/predict

Predict

Prediction Endpoint

New tab

Iris Classification API - Swagger UI

localhost:8000/docs#/default/predict_predict_post

POST

/predict

Predict

Predict the Iris flower class based on input features.

Args: iris_features: IrisFeatures object containing sepal_length, sepal_width, petal_length, and petal_width

Returns: Dictionary with predicted class, class name, and probabilities

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

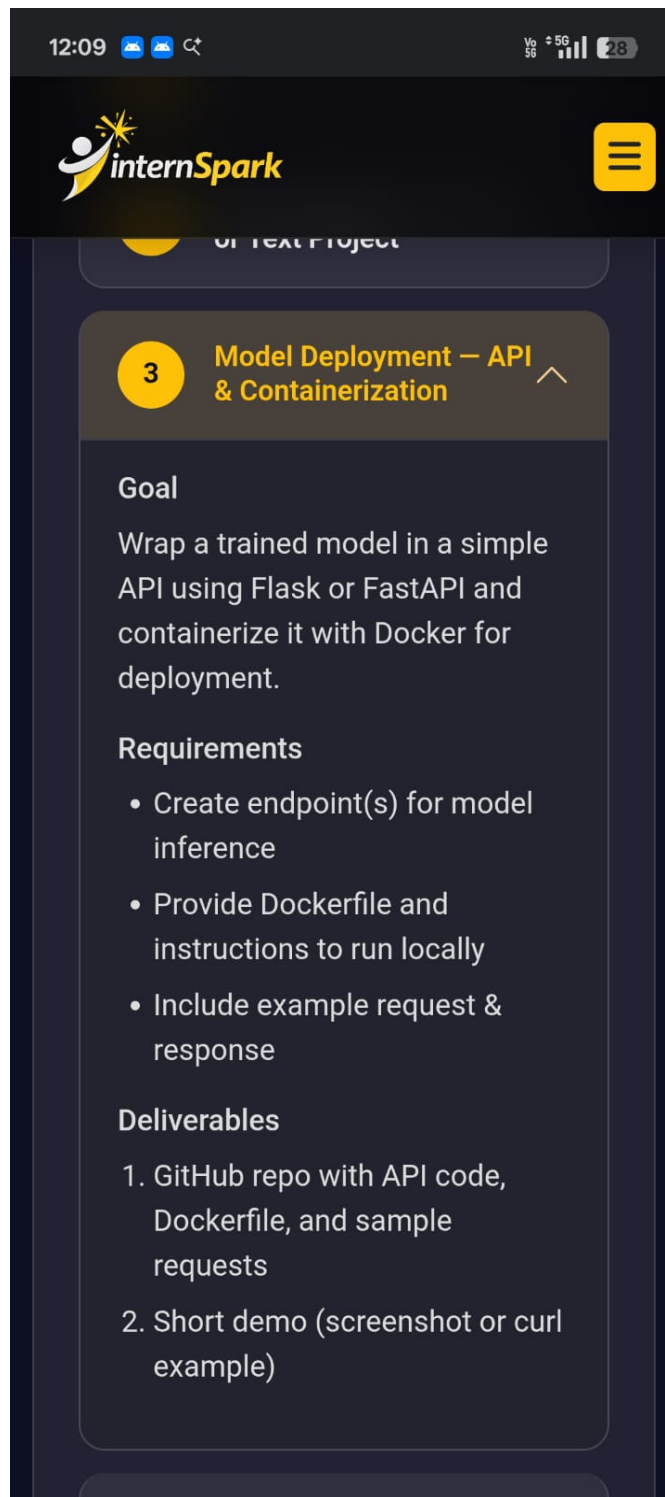
Schema

```
{
  "sepal_length": 0,
  "sepal_width": 0,
  "petal_length": 0,
  "petal_width": 0
}
```

Responses

Code	Description	Links
200	Successful Response	No links

Task Requirements



Sample Request

```
{  
  "sepal_length":5.1,  
  "sepal_width":3.5,  
  "petal_length":1.4,  
  "petal_width":0.2  
}
```

Deliverables

GitHub repository, Dockerfile, API code, screenshots, and sample requests/responses.

Conclusion

The project successfully demonstrates machine learning model deployment and API containerization concepts.